GDB for Linux Database Debugging: A Deep-Dive Reference for PostgreSQL and MySQL/MariaDB Server Backends

TL;DR

- **GDB is a ptrace(2)-based, stop-the-world debugger** that controls target ("inferior") processes by injecting `INT3` (0xCC) software breakpoints or programming the x86 DR0–DR7 debug registers, and reads DWARF debug info emitted by `-g`/`-ggdb`/`-g3`. For databases, its defining property is that **attaching stops the process** — which is exactly why it is dangerous on production PostgreSQL, where a paused backend holding an LWLock or spinlock can stall the entire cluster. The PostgreSQL wiki's own workflow says: after `sudo gdb -p PID` and setting logging, "*execution of the backend is still paused. It can even hold up other backends, so I recommend that you tell it to resume executing normally with the `cont` command.*"
- **PostgreSQL (process-per-connection) and MySQL/MariaDB (thread-per-connection) require fundamentally different GDB workflows.** For Postgres you find one backend PID (`SELECT pg_backend_pid()`, `pg_stat_activity`) and attach to that process; the postmaster forks backends, so multi-process debugging (`set follow-fork-mode`, `set detach-on-fork off`) matters. For mysqld/mariadbd you attach to one multithreaded process and pivot with `thread apply all bt full`, then locate the connection thread and read its `THD` (`print thd->query_string`).
- **The safe production pattern is non-destructive core capture, not interactive attach.** Use `gcore PID` (which "*generate[s] core dumps of one or more running programs*" while "*the program remains running without any change*") or `kill -ABRT`/`--core-file`, then analyze the core offline with `gdb program core` + `bt full` and `thread apply all bt full`. Reserve live attach for hangs where you interrupt→`bt`→`cont` repeatedly to sample, and reserve time-travel (Mozilla **rr**, whose "*recording and replay slowdown is less than a factor of two*") for reproducible development bugs. GNU Project arxiv

---

Key Findings

1. **Architecture.** GDB uses the Linux `ptrace(2)` API (PTRACE_ATTACH/SEIZE, GETREGS/SETREGS, PEEKTEXT/POKETEXT, CONT) to inspect and manipulate a target's registers, memory, and file descriptors. Software breakpoints work by saving the original byte and overwriting it with the single-byte `INT 3` (0xCC) trap instruction; hardware breakpoints/watchpoints use the four x86 address registers DR0–DR3 with type/length control in DR7 (only 4 hardware slots on x86; 2 on typical ARM) and are the only way to break on data access without single-stepping. arxiv
2. **Debug info & symbols.** Useful traces require DWARF debug info. Distro binaries strip it into separate `.debug`/debuginfo packages keyed by build-id; without them, backtraces degrade to `0x00686a3d in ?? ()`, which the PostgreSQL wiki explicitly calls out as "*completely useless for debugging … Do not bother collecting such backtraces.*" `debuginfod` can fetch matching symbols on demand. postgresql
3. **The production-safety caveat is the single most important database-specific point.** A backend paused under GDB while holding a shared-memory lock (LWLock, spinlock, buffer pin) blocks every other backend that needs that lock. Postgres's own hackers debug LWLock deadlocks this way and note the hazard ("*I messed up the gdb session causing the postmaster to SIGKILL all the children*"). Spinlocks (`s_lock`/`tas`) provide no deadlock detection and no monitoring; freezing a spinlock holder is especially damaging. Postgres Professional Postgres Professional
4. **PostgreSQL has first-class GDB integration**: `pprint()`/`print()` node dumpers callable from GDB, the `errfinish` breakpoint idiom for trapping errors, `src/tools/gdbinit`, and Python pretty-printers (`gdbpg.py`'s `pgprint`, `pg_pretty_printer`, Bitmapset printers). Postgres 13+ works with **rr**; v14+ exposes `pg_backend_memory_contexts`/`pg_log_backend_memory_contexts` so you no longer always need GDB for MemoryContext inspection. PostgreSQL Wiki
5. **MySQL/MariaDB expose the query and schema through the `THD` structure** (`print thd->query_string`), print their own numeric stack trace via a crash handler (`resolve_stack_dump` against the symbol table), and ship a `.gdbinit` recommending `handle SIGUSR1/SIGUSR2 … nostop noprint`. MariaDB provides official debug containers (`quay.io/mariadb-foundation/mariadb-debug`) and a canonical batch command for full traces. MariaDB
6. **GDB perturbs the target (all-stop); sampling profilers (perf) do not.** This is the core reason perf/eBPF are preferred for live production diagnosis and GDB/gcore for post-mortem and development.

Details

1. What GDB is and its architecture

Overview & history. GDB, the GNU Project Debugger, is the Free Software Foundation's portable source-level debugger, distributed under

GPLv3, supporting Ada, C, C++, Objective-C, Fortran, Go, Rust and more, running native or remote across most UNIX variants and Windows. Per the GNU project page, GDB lets you "*see what is going on 'inside' another program while it executes — or what another program was doing at the moment it crashed.*" It is developed in the combined **binutils-gdb** git tree hosted at sourceware.org (`git clone ssh://sourceware.org/git/binutils-gdb.git`); GDB and GNU binutils share this repository and much low-level infrastructure (BFD, libiberty, opcodes) as part of the GNU toolchain alongside GCC. Release branches are cut per major version (e.g. the `gdb-8.0-branch`), followed by point releases (7.12 → 7.12.1). Reversible ("time-travel") debugging landed in **gdb 7.0 (October 2009)**. The canonical documentation is the manual *"Debugging with GDB"* at `sourceware.org/gdb/current/onlinedocs/gdb.html/`. GDB is distributed as distro packages (`apt install gdb`, `dnf install gdb`) or built from source from the binutils-gdb tree. uspto gnu

ptrace core. Conventional debuggers "*use kernel ptrace support to interact with the kernel to debug an application.*" Through ptrace, "*one process can control another, enabling the controller (i.e., the debugger) to inspect and manipulate the internal state of its target*," including "*its file descriptors, memory, and registers.*" Key operations: `PTRACE_ATTACH`/`PTRACE_SEIZE`, `PTRACE_GETREGS`/`SETREGS`, `PTRACE_CONT`, `PTRACE_PEEKTEXT`/`POKETEXT`. GDB waits on the target via `wait()` to learn when it stops on a debug event (breakpoint/signal). uspto + 2

Breakpoints. *Software breakpoints*: GDB "*save[s] a specified instruction in the program to replace it with an exception-triggering instruction … in x86-64 the exception-triggering instruction is usually a special single-byte instruction (i.e., INT 3)*" (opcode 0xCC); after the trap fires and the callback runs, "*the original instruction is written back and executed.*" *Hardware breakpoints/watchpoints*: implemented "*at CPU-level by using dedicated debug registers (e.g., DR0–DR3 in the x86-64 architecture …). When the program counter matches a value and a set of conditions are met (determined by the DR7 register …), a debug exception is triggered.*" DR0–DR3 hold four addresses; per-address R/W0–R/W3 and LEN0–LEN3 fields in DR7 select access type (execute / write / read-write) and size; L0–L3/G0–G3 enable them; DR6 is the status register reporting which breakpoint fired. Because there are only four such registers, hardware watchpoints are a scarce resource; when exhausted GDB falls back to slow software watchpoints (single-stepping and re-checking). arxiv + 3

Remote protocol & gdbserver. For remote/cross debugging, GDB and a "*debugging stub*" (or `gdbserver`) "*communicate via a message-based protocol that contains commands to read and write memory, query registers, run the program.*" `gdbserver` runs on the target, GDB connects with `target remote host:port`. Notably, **rr implements a gdb backend** — replay presents itself to GDB as a remote target. uspto uspto

Inferiors & record/replay. GDB calls each debugged process an "*inferior*"; multi-process debugging uses `info inferiors`/`inferior N`. Process record/replay logs execution so you can step backwards; the hardware-based reverse debugging in stock GDB is slow and single-threaded, which is why rr is preferred for heavy workloads. postgresql

2. Core workflow and commands

- **Start / attach / core:** `gdb --args postgres -D /path/to/data`; attach with `sudo gdb -p PID` (or `attach PID`); post-mortem with `gdb /path/to/postgres /path/to/core`. GDB prints the current frame on attach, e.g. `0xb7c73424 in __kernel_vsyscall ()`. postgresql
- **Breakpoints:** `break func` / `tbreak` (temporary) / conditional `break errfinish if errordata[errordata_stack_depth].elevel >= 20` / `commands` (auto-run commands at a breakpoint). **Watchpoints:** `watch expr` (write), `rwatch` (read), `awatch` (any); prefer `watch -l expr` under rr so reverse execution isn't slow/buggy. **Catchpoints:** `catch fork`, `catch exec`, `catch syscall`. PostgreSQL Wiki postgresql
- **Execution control:** `run`, `continue`/`cont`, `next`, `step`, `stepi`/`nexti`, `finish` (run to caller — Postgres hackers use `fin` repeatedly to see if a stuck backend makes progress), `until`, `return`. Postgres Professional
- **Stack:** `backtrace`/`bt`, `bt full` (with locals — the wiki now recommends `bt full` over `bt`), `frame N`, `up`, `down`, `info frame`, `info args`, `info locals`. postgresql
- **Data:** `print`/`p`, `ptype`, `whatis`, `x` (examine memory), `display`, `set variable`, and `call` (invoke a function in the inferior — the basis of calling `pprint` in Postgres). `dump binary memory file start end` extracts raw memory (e.g. dumping an 8 KiB Postgres page: `dump binary memory /tmp/dump_block.page origpage (origpage + 8192)`). postgresql
- **Source / info:** `list`, `directory`; `info threads`, `info registers`, `info sharedlibrary`, `info proc mappings`, `info inferiors`, `info pretty-printer`.

3. Advanced features

- **Multi-threaded:** `thread apply all bt` / `thread apply all bt full`, `thread N`, `set scheduler-locking on|step|off`, `set non-stop on` (let other threads run while one is stopped) vs. default all-stop. GDB "*cannot single-step all threads in lockstep*," complicating race analysis. arxiv
- **Multi-process / fork (critical for Postgres):** `set follow-fork-mode parent|child`, `set detach-on-fork off` (debug both), `set schedule-multiple on`, `catch fork`/`exec`. The postmaster forks each backend, so to catch a child from birth you keep both inferiors and switch with `inferior N`.
- **Reverse / time-travel:** `record`, `reverse-continue`, `reverse-step`, `reverse-next`; in practice, use rr and `reverse-continue` to a breakpoint.

- **Python API:** custom commands, convenience functions, frame filters, and **pretty-printers** registered via `gdb.pretty_printers.append(func)` or `RegexpCollectionPrettyPrinter`; GDB's auto-load mechanism loads `<binary>-gdb.py` only for the matching program (used deliberately by pg pretty-printers to avoid `Node` type-name collisions). Undo + 2
- **TUI:** `tui enable` / `Ctrl-x a`, `layout src|asm|regs|split` for a curses source/assembly/register view.
- **`.gdbinit`:** per-user or per-project init file; commit a project `.gdbinit` so a team shares macros/pretty-printers. Undo
- **Checkpoints:** `checkpoint`/`restart`/`delete checkpoint` (fork-based snapshots; also supported through rr for finer-than-event-number granularity). postgresql

4. PostgreSQL-specific debugging (primary focus)

Finding the backend. Get the PID with `SELECT pg_backend_pid()` in the target psql session, or via `pg_stat_activity`/`pg_locks` (join on `pid`), or `top` for a CPU-bound backend. Confirm the executable via `/proc/$pid/exe`. postgresql

Attach and the production hazard. `sudo gdb -p PID`; then:

```
(gdb) set pagination off
(gdb) set logging file debuglog.txt
(gdb) set logging on
(gdb) cont
```

The wiki is explicit that the backend is paused on attach and "*can even hold up other backends*," so you `cont` immediately. The deeper danger: if that backend holds an **LWLock** (WALInsertLock, buffer-mapping, lock-manager partition — there are 16 lock-manager partitions), a **spinlock** (`s_lock`/`tas`, no deadlock detection, no monitoring), or a **buffer pin**, every backend waiting on it hangs for as long as GDB holds the process — potentially the whole cluster. **Never run interactive breakpoints on a production primary**; prefer `gcore` (below) or a replica/repro. pganalyze Postgres Professional

Trapping errors and crashes.

- Error origin: `b errfinish` then `cont`; provoke the error from psql. `errfinish` traps *all* ereport/elog levels (NOTICE/LOG/ERROR), so filter by `elevel >= 20` (ERROR/FATAL/PANIC) or set `client_min_messages`/`log_min_messages` to reduce noise. postgresql
- Reproducible crash: attach, `cont`, trigger crash — GDB stops automatically at the fault; `bt` (or `bt full`), then `cont`/`quit`. postgresql

Inspecting internal structures.

- Call Postgres's own dumpers: `call pprint(plannedstmt)` (verbose) or `call print(node)` (short). Output goes to the server log/stderr. Works on any `Node*` — `Query*`, target lists, `PlannedStmt`, `EState`, `ExprContext`, `TupleTableSlot`, `MemoryContext`. PostgreSQL Wiki
- Pretty-printers make nested `Plan`/`PlannedStmt` trees readable instead of manually chasing `plan->planTree->lefttree->lefttree` pointers: load `gdbpg.py` (`tvondra/gdbpg`) and use `pgprint`; or `askyx/pg_pretty_printer` (400+ node types, auto-load); or a dedicated **Bitmapset** printer that decodes `{nwords, words}` into `PGBitmapset ([1,2,…])` and works on core dumps even when Postgres isn't running. Pgaddict Jnidzwetzki
- Query string of a running/crashed backend: read `ActiveSnapshot`/`debug_query_string` or the current `QueryDesc`; from a `Query*`, `pprint` it.

Core dumps. Enable with `ulimit -c unlimited` (in the startup script) and a non-clobbering pattern via `/proc/sys/kernel/core_pattern` (e.g. `core.%p.sig%s.%ts` or `core.%e.%p.SIG%s.%t`); verify per-process with `/proc/$pid/limits` ("Max core file size" non-zero). Because Postgres uses large shared memory, temporarily reduce `shared_buffers` to avoid multi-GB, system-stalling cores, and set the **coredump_filter** (`echo 0x33 > /proc/$pid/coredump_filter`) / GDB `set use-coredump-filter off` when you actually need shared memory in the dump. Test with `kill -ABRT <backend_pid>`. Analyze: `sudo -u postgres gdb -q /usr/lib/postgresql/NN/bin/postgres /path/core` then `bt full`. The wiki's own example shows a WAL writer core resolving to `WalWriterMain → AuxiliaryProcessMain` once symbols are installed — and the same trace as `?? ()` without them. postgresql + 5

Starting Postgres under GDB (development). Stop the server, `gdb --args postgres -D data`, then `set detach-on-fork off`, `set schedule-multiple on`, `handle SIGUSR1/SIGUSR2 noprint nostop`, define macros so Postgres macros evaluate (`macro define __builtin_offsetof(T,F) ((int) &(((T *) 0)->F))`), `macro define __extension__`), and `run`. Use `info inferiors`/`inferior NUM` to switch between postmaster and backends. The wiki warns this is "*still quite fragile, so don't expect to be able to do this in production.*" postgresql + 2

Build flags. Per the Postgres Developer FAQ, when developing C code you should "*ALWAYS work in a build configured with the `--enable-cassert` and `--enable-debug` options*"; asserts add sanity checks, `--enable-debug` adds symbols. Optimized builds (`-O2`) inline functions

and drop frame pointers, producing `<optimized out>` for variables and missing stack frames; build with `CFLAGS="-Og -g3"` or `-O0` for reliable local/argument visibility. `pg_config` reports the configure/CFLAGS used. PostgreSQL Wiki

printf vs GDB in the community. Postgres relies heavily on `elog`/`ereport` and `backtrace_functions`/`backtrace_on_error` GUCs (set `backtrace_functions='typenameType'` to attach a backtrace when a specific function raises). These need `-rdynamic` for symbol names (Postgres normally isn't built with it, so `addr2line -e postgres <addr>` resolves addresses); static function names still won't appear. GDB tracepoints are "*much more powerful than … perf … with the tradeoff that they're much more intrusive.*" Blogger + 2

Extensions. Attach to the backend, `break my_extension_func`, run the SQL that calls it; build the extension with `-O0 -g` via its Makefile/PGXS `CFLAGS`. Watch for `CLOBBER_FREED_MEMORY` builds filling freed memory with `0x7f` bytes. PostgreSQL Wiki

rr for Postgres (13+). `rr record postgres -D data …` records a full `make installcheck` "*not much slower than … a regular debug build … much faster than Valgrind.*" Replay to an event with `rr replay -M -g <event>` or to a backend's fork with `rr replay -M -f <pid>`; use `when`, `checkpoint`, and `reverse-continue`. Chaos mode (`rr record -h`) helps reproduce races. postgresql + 3

5. MySQL / MariaDB-specific debugging

Threaded architecture changes the workflow. mysqld/mariadbd is one process, thread-per-connection (`handle_one_connection` → `do_command` → `dispatch_command` → `mysql_parse` → `mysql_execute_command`). You attach once and use thread-centric commands. Attach in a container: `podman exec -ti --user mysql mdb105 gdb -p 1` (needs `--cap-add CAP_SYS_PTRACE`). Slideshare MariaDB

Core & full traces. Start with `--core-file` to dump on SIGSEGV; open with `gdb /usr/sbin/mariadbd /var/lib/mysql/core.NNN`. MariaDB's canonical batch command:

```
gdb --batch --eval-command="set print frame-arguments all" \
    --eval-command="thread apply all bt full" \
    /usr/sbin/mariadbd /var/lib/mysql/core.932 > mariadbd_full_bt_all_threads.txt
```

Interactive session: `set logging file /var/lib/mysql/gdb_output.txt`, `set pagination off`, `set logging on`, then `thread apply all bt full`, `info threads`, `info regs`. MariaDB

Extracting the query, DB and table. Read the stack bottom-up, find the frame for `mysql_execute_command`/`mysql_parse`, `frame N`, then `print thd->query_string` — e.g. `$1 = {string = {str = 0x… "SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.Columns where TABLE_NAME='my_workflow' …", length=111}, cs=…}`. The `THD` also yields the active database and connection info; this is often enough to correlate with logs or file a bug (Percona's workflow on RHEL/AlmaLinux uses a UBI9 container with matching Percona debug RPMs — the binary and core must match exactly). Percona + 3

MySQL's own crash handler. On a fatal signal, `handle_fatal_signal` prints a numeric stack trace; if unresolved you follow `resolve_stack_dump` against the symbol table (the docs referenced by the crash message). A resolved trace is "*much more helpful.*" MySQL

Debug builds & the DBUG package. Build with `WITH_DEBUG` to get a debug binary (historically `mysqld-debug`), which enables the built-in **DBUG** trace facility (Fred Fish's package): `DBUG_ENTER`/`DBUG_RETURN`/`DBUG_PRINT` macros produce function-call traces controlled at runtime with `--debug` (e.g. `-#d:t:o,/tmp/mysqld.trace`) — MySQL documents this under "Creating Trace Files" and "The DBUG Package." The `--gdb` option installs a SIGINT handler and disables stack tracing/core handling so you can `^C` into GDB and set breakpoints; run mysqld with `--skip-stack-trace` under GDB so GDB catches segfaults itself. Because GDB doesn't free memory for old threads, set `thread_cache_size` high (≈`max_connections+1`, or just `--thread_cache_size=5`) when doing many connections. mysql mysql

Recommended `.gdb` file (ship in cwd): `handle SIGUSR1/SIGUSR2/SIGWAITING/SIGLWP nostop noprint`, `handle SIGPIPE/SIGALRM/SIGHUP nostop`, `set print sevenbit off`. Newer LOCK_ORDER tooling and InnoDB-specific structures round out server debugging. MariaDB Foundation publishes debug containers (`quay.io/mariadb-foundation/mariadb-debug:10.5`) so you can run `gdb --args mysqld` with symbols and correct permissions. mysql MariaDB

6. Practical considerations & best practices

- **Stop-the-world vs sampling.** GDB (all-stop) freezes the target, perturbing timing and — in a database — holding locks. Sampling profilers (`perf`) and eBPF are non-intrusive; use them for live production hotspot/latency diagnosis and reserve GDB for post-mortem and reproducible bugs.
- **ptrace_scope / Yama.** Attach failures ("*Could not attach to process … Operation not permitted*") usually mean `/proc/sys/kernel/yama/ptrace_scope` is `1` (the Ubuntu/Debian default, "restricted ptrace": only descendants attachable). Fix live with `sysctl -w kernel.yama.ptrace_scope=0`, persist in `/etc/sysctl.d/10-ptrace.conf`; or attach as root / grant `CAP_SYS_PTRACE` (the scope "*is ignored when a user has CAP_SYS_PTRACE*"). Value `2` = admin-only; `3` = no attach at all (irreversible until reboot). In

containers, ptrace is blocked by the default seccomp profile — run with `--cap-add CAP_SYS_PTRACE` (and adjust seccomp). A debuggee can also opt in via `prctl(PR_SET_PTRACER, pid, …)`. Note user namespaces can inadvertently weaken Yama. (ttias.be + 7)

- **Optimized/release binaries.** Expect `<value optimized out>`, missing frames, inlined functions and tail calls. Install matching debuginfo (build-id–keyed `.debug` files; `debuginfo-install pkg`, Debian `find-dbgsym-packages`/`list-dbgsym-packages.sh`, Fedora debuginfo). Always install libc debug symbols plus the DB server and its libraries. (postgresql)
- **Common pitfalls:** wrong executable vs core mismatch (even a minor patch difference invalidates analysis); `?? ()` frames = missing symbols; `No such file or directory` for source = source tree not present (get the matching tagged source); NPTL/`LD_ASSUME_KERNEL` quirks noted in MySQL docs for old GDBs. (mysql)
- **LLDB (LLVM debugger)** is the main alternative: a cross-platform debugger backed by Apple/Google, closely tied to Clang/LLVM, default on macOS. It offers similar ptrace-based control and a Python API; on Linux GDB has broader distro/debuginfod integration and is the de-facto choice for Postgres/MySQL server debugging, while LLDB is common in Clang-centric and macOS environments. (arxiv)

7. Related tools and ecosystem

- **gcore** — the safest database tool: `gcore [-a] [-o prefix] PID` produces a core "*equivalent to one produced by the kernel*," after which "*the program remains running without any change.*" It still pauses the process briefly (proportional to RSS and disk speed) and invokes GDB via ptrace, so it's subject to `ptrace_scope`. `-a` dumps all mappings (disables coredump-filter). Analyze the core at leisure — ideal for capturing a snapshot of a hung backend without holding locks interactively. (GNU Project + 2)
- **gdbserver** — remote/cross debugging stub. **pstack/gstack** — quick one-shot backtrace of a live process (or core): `gstack PID` (a GDB wrapper) / `pstack core.PID`. (Veritas)
- **IDE / frontends:** VS Code (C/C++ extension over GDB/MI), CLion, Emacs (`M-x gdb`), DDD, and `gdbgui` (browser frontend). Eclipse CDT standalone debugger works with Postgres. (PostgreSQL Wiki)
- **Valgrind** — memcheck/helgrind for memory and threading errors (very heavy; ~10–50× slowdown). **rr (Mozilla)** — record-and-replay reverse debugging built on ptrace + seccomp-bpf + perf counters, "*recording and replay slowdown … less than a factor of two*," in daily use on Firefox/Chromium/QEMU/Wine; it sequentializes threads (so parallel workloads pay more) and, per its authors, "*cannot be implemented on ARM CPUs.*" UndoDB is a commercial equivalent. rr's determinism (stable pointers/PIDs across replays) makes it the best tool for hard-to-reproduce Postgres/MySQL race conditions in development. (arxiv + 4)

Recommendations

Stage 0 — Prepare before you need it. Install matching debuginfo (server + libc + libraries; verify with a trial `bt` — reject any trace full of `?? ()`). Set a non-clobbering `core_pattern` (`core.%e.%p.SIG%s.%t`), enable `ulimit -c unlimited` in the DB startup unit, and know your `ptrace_scope` value. For dev/staging, build Postgres with `--enable-debug --enable-cassert` and `CFLAGS="-Og -g3"` (or `-O0`), and MySQL/MariaDB with `WITH_DEBUG`. Load `gdbpg.py` / a Bitmapset pretty-printer in your `.gdbinit` (via auto-load, not globally, to avoid `Node` name collisions).

Stage 1 — Live production incident (hang or runaway backend). Do **not** set interactive breakpoints. First use `perf`/eBPF and `pg_stat_activity.wait_event`/`pg_locks` (or MySQL `performance_schema`/`SHOW PROCESSLIST`) to characterize the wait. If you must go deeper, run `gcore PID` to snapshot without holding locks, or attach and immediately `cont`, then interrupt→`bt`→`cont` a few times to build a sampled picture (the wiki's "representative traces" method). For MySQL, `thread apply all bt full` on the gcore. **Threshold to escalate to interactive attach:** only on a replica, a non-critical standby, or after failover — never on the sole primary while it holds shared-memory locks. (postgresql)

Stage 2 — Post-mortem of a crash. Locate the core (Postgres data dir, or `coredumpctl list` with systemd). Confirm the binary matches exactly. Run `gdb <binary> <core>`, `set pagination off`, `bt full`, and `thread apply all bt full`. For Postgres, `call pprint(...)` on relevant nodes / use pretty-printers; for MySQL, `frame N` to the executor frame and `print thd->query_string` to recover the offending SQL, database, and table. (MariaDB)

Stage 3 — Reproducible development bug. Use rr: `rr record` the failing test/`make installcheck`, then `rr replay` with `reverse-continue`, `watch -l`, and `checkpoint` to zero in. This is dramatically faster than Valgrind and immune to the perturbation problems of live GDB.

Benchmarks that change the plan: if `bt` shows `?? ()` → stop and install debuginfo. If variables show `<optimized out>` → rebuild with `-Og`/`-O0`. If attach fails "Operation not permitted" → check/lower `ptrace_scope` or add `CAP_SYS_PTRACE`. If shared_buffers is many GB → shrink it before enabling cores, or filter shared memory out of the dump. If the incident is on the sole primary → switch to `gcore`/perf/`pg_log_backend_memory_contexts` rather than interactive GDB.

Caveats

- **The lock-freeze hazard is real but its blast radius depends on which lock is held.** A backend paused while merely `epoll_wait`/idle harms nothing; one holding a WALInsertLock, a lock-manager partition, a buffer-mapping LWLock, or a spinlock can stall many or all backends. You often cannot know in advance which lock a target holds — hence the strong bias toward `gcore` and non-intrusive tools on production. Treat every production attach as potentially cluster-affecting.
- **Some GDB behaviors are version- and build-dependent.** Whether function arguments appear in traces depends on frame pointers, optimization level and DWARF version; the exact set of `set follow-fork-mode`/`schedule-multiple` semantics and rr syscall coverage vary by version. The GNU project's own front page is stale (last major content dated 2017); rely on the versioned manual at `sourceware.org/gdb/current/onlinedocs/` for current command semantics.
- **MySQL's `LD_ASSUME_KERNEL` advice is legacy** (old NPTL-era guidance) and generally irrelevant on modern glibc; treat it as historical.
- **rr constraints:** x86 only (no ARM), requires specific Intel PMU/perf and seccomp-bpf support, serializes threads, and produces large traces; older distro rr may lack syscalls Postgres uses (`sync_file_range`) — workaround by disabling flush GUCs. It is a development tool, not a production one. `postgresql`
- **Container/cloud managed databases** (RDS, Aurora, Cloud SQL) generally deny ptrace and shell access entirely; GDB-based techniques here are limited to self-managed instances or vendor-provided core dumps. Aurora is PostgreSQL-*compatible* but not stock Postgres, so internal-structure offsets and pretty-printers may not match.
- I was unable, within this session, to independently re-verify a few finer points against primary sources — notably the exact current default GDB release cadence/version number and the precise contents of `src/tools/gdbinit` in the latest Postgres tree — because search rate limits truncated late queries; the workflow substance above is corroborated by the PostgreSQL wiki, MySQL/MariaDB official docs, kernel.org (ptrace/Yama), and the GDB manual, but treat specific version numbers as approximate and confirm against the versioned manual and the current Postgres source tree before quoting them.